

INPUT & OUTPUT en Angular

L'essentiel de la communication entre composants (parent ↔ enfant) · API moderne à base de signaux

Idée clé : un composant enfant ne connaît pas son parent. La communication se fait par un « contrat » : les **INPUT** font descendre des données *du parent vers l'enfant*, les **OUTPUT** font remonter des événements *de l'enfant vers le parent*.

▼ INPUT (entrée)

Le parent **envoie une donnée** à l'enfant. L'enfant la *reçoit* et l'affiche.

Déclaré avec `input()` dans l'enfant.

Parent → Enfant (données)

▲ OUTPUT (sortie)

L'enfant **signale un événement** (clic, action) au parent, qui réagit.

Déclaré avec `output()` dans l'enfant.

Enfant → Parent (événements)

1. INPUT — recevoir une donnée

Dans l'enfant — on déclare la propriété d'entrée :

```
import { Component, input } from '@angular/core';

export class PlatCard {
  plat = input.required<Plat>(); // obligatoire
  // plat = input<Plat>();        // optionnel
  // remise = input(0);           // avec valeur par défaut
}
```

Dans le parent — on passe la valeur entre crochets `[ ]` (liaison de propriété) :

```
<app-plat-card [plat]="unPlat" />
```

Utilisation dans le template de l'enfant — c'est un signal, on l'appelle avec `()` :

```
<h3>{{ plat().nom }}</h3>
<p>{{ plat().prix }} FCFA</p>
```

2. OUTPUT — émettre un événement

Dans l'enfant — on déclare la sortie, puis on émet avec `.emit()` :

```
import { Component, output } from '@angular/core';

export class PlatCard {
  ajouter = output<Plat>(); // flux d'événements typé
}
```

```
<!-- template de l'enfant -->
<button (click)="ajouter.emit(plat())">Ajouter</button>
```

Dans le parent — on écoute l'événement entre parenthèses `()` ; `$event` contient la valeur émise :

```
<app-plat-card [plat]="unPlat"
  (ajouter)="ajouterPlat($event)" />
```

```
// dans la classe du parent
ajouterPlat(plat: Plat) {
  // réagit à l'événement remonté par l'enfant
}
```

3. Tableau récapitulatif

	INPUT	OUTPUT
Rôle	Recevoir une donnée	Émettre un événement
Sens	Parent → Enfant	Enfant → Parent
Déclaration	<code>input()</code> / <code>input.required()</code>	<code>output()</code>
Syntaxe (parent)	<code>[prop]="valeur"</code> (crochets)	<code>(event)="methode(\$event)"</code> (parenthèses)
Côté enfant	Lecture : <code>maProp()</code>	Envoi : <code>maSortie.emit(valeur)</code>
Analogie	Arguments d'une fonction	Notification / callback

4. À retenir

- **Crochets = données qui descendent, parenthèses = événements qui remontent.**
- Avec la nouvelle API, un input est un **signal** : on le lit avec `()` et l'affichage se met à jour tout seul.
- `input.required()` rend l'entrée obligatoire (erreur si le parent ne la fournit pas).
- Un `output()` transporte une valeur typée récupérée côté parent via `$event`.
- Cette séparation rend les composants **réutilisables** et **découplés** : l'enfant ne dépend pas d'un parent précis.

Exemple concret (Délices de Douala) : PlatCard reçoit un `plat` (INPUT) et émet `ajouter` (OUTPUT) au clic ; LigneCommande reçoit une `ligne` (INPUT) et émet `incrémenter` / `decrémenter` (OUTPUT). Le parent Commandes orchestre le tout.